Tradeoff Analysis

Every design involves giving something up. This document explains what this system gives up, what it gets in return, and when I would decide differently. The design itself is in the separate System Design document.

The rule behind most of these choices: **in healthcare data, a wrong answer is worse than a slow answer or a missing one.** A wrongly merged patient or a silently changed value can reach a clinician; a duplicate or a quarantined record only costs someone time.

1. Latency vs accuracy and reliability

Decision	Faster option given up	Why
Single records sync, files async	Processing files inside the request	A single record takes milliseconds, so the sender gets an answer at once. A 100,000-row file would hit HTTP timeouts; as a background job it survives restarts and reports progress
Match verification is async	Confirming matches instantly	A candidate match waits as `pending_verification` until the verifier answers. Nothing treats the two records as one person in the meantime. A delay of seconds is acceptable; a wrong merge is not
Ambiguous data is quarantined, not guessed	Higher throughput, fewer records waiting for a person	`03/04/1990` could be two different birthdays. Guessing would be fast and sometimes wrong, invisibly. Quarantining costs a person a minute but never stores a wrong date
A savepoint per row	One commit per chunk without savepoints	Savepoints add a little overhead per row, but one bad row then rolls back alone instead of undoing 499 good ones
Chunks of 500 rows	Larger chunks with fewer commits	Larger chunks are slightly faster, but a crash repeats more work and transactions hold locks longer. 500 balances the two
Stale jobs taken over after 300 s	Faster crash recovery with a shorter timeout	Too short, and a slow but healthy worker loses its job, so the work runs twice (safe, thanks to idempotency, but wasteful). Too long, and recovery is slow. Five minutes suits batch uploads, and it is configurable
Retries for transient errors only (1 s, 4 s, 16 s)	Retrying everything	A bad date fails the same way every time, so retrying it wastes work. A lost connection may succeed seconds later. Retrying the whole chunk outside the transaction also avoids holding locks while waiting
Verifier timeout of 20 s, failing closed	Waiting longer for an answer	If the verifier is slow or down, the pair goes to human review rather than blocking the pipeline or being merged unchecked

2. Cost vs quality

Where the system spends more to get more

Cost	What it buys
Storing everything: raw payloads, every version, and a log row for every changed field: several times the storage of clean output alone	Any value can be traced to its source and the exact rule version that produced it, and everything can be reprocessed after a bug fix. Storage is cheap; an untraceable clinical error is not

Cost	What it buys
`Decimal` arithmetic instead of floats	Slightly slower, but 154 lb becomes exactly 69.85 kg, not 69.85322497999999
Human review of uncertain matches. In the evaluation, 66 pairs needed a person, for 938 patient records	The real operational cost of the system is reviewer time. Lowering the thresholds would review fewer pairs but merge more wrongly. Raising them is safer but costs more people-hours. The current setting produced zero wrong merges

Where the system saves cost

Decision	Saving	Quality given up
Postgres as the job queue (`SKIP LOCKED`)	No message broker to run, pay for or monitor	Polling adds a little database load; a managed queue scales further (section 3)
One database for everything	One system to back up, secure and operate	All load on one instance until it needs splitting
Blocking in matching (compare only records sharing a birth year or family-name prefix)	24,354 comparisons instead of 439,453 in the evaluation, 94% fewer	A true pair whose birth year *and* first three letters of the surname both contain typos is never compared. Rare, and accepted
Rule-based verifier by default	Zero AI cost, no external calls, fully repeatable	Less flexible reasoning on unusual cases
Small model if the LLM verifier is enabled (`claude-haiku-5-5`)	The task is narrow: about 12 categorical facts in, one of three answers out. A small model is fast and inexpensive, and a larger one would add cost and delay without better answers	Less capable on complex reasoning, which this task doesn't need
AI only on candidate matches	In the evaluation, only about 2% of compared pairs reached verification, so AI cost scales with matches, not with records	None: every automatic match is still checked
Rate limits in process memory	No Redis needed	With several API instances, each counts separately
Local Docker setup instead of a cloud deployment	No hosting cost; reviewers run the full system with one command. A single-server production configuration is included	No live URL, which a take-home on synthetic data doesn't need

AI usage: why so little, and only as a veto

AI could make several parts of this system "smarter": guessing date formats, filling missing fields, deciding matches. I deliberately used it in only one place. It is the final check on candidate matches that the scoring rules have already proposed. It can block one of those candidates, but it can never promote a pair the rules didn't propose. The reasons:

- **Safety:** a wrong veto costs a reviewer a few minutes. A wrong approval lets through a match the rules proposed, which leaves the system exactly as accurate as the rules alone. The model can add caution, but it can't make matching less safe than the rules already are.
- **Repeatability:** healthcare decisions must be explainable and reproducible. Rules are; model outputs can vary between runs. So rules decide, and the model only adds caution.

- **Privacy:** the model sees comparison facts such as `birth_date: agree`, never names, dates or identifiers.
- **Cost:** it runs on a small fraction of pairs and is off by default.

3. Simplicity vs scalability

Area	Built now (simple)	When to change	Then
Services	One codebase: API + worker	Separate teams, or very different scaling needs	Split ingest, normalization and matching into services
Job queue	Postgres table with `SKIP LOCKED`	Polling becomes a noticeable load on the database	SQS; the job runner is already a separate module
Database	One Postgres	Hundreds of millions of rows	Partition by org and month; archive old raw payloads to object storage; read replicas
Tenancy	Shared tables with forced row-level security	A customer contract requires physical separation	Schema or database per large customer
Matching	Hand-set weights, blocking	Real labelled data is available	Learned weights, or a dedicated MPI service
Rate limits	In-memory per process	More than one API instance	Shared counters in Redis
Authentication	Hashed API keys with roles	Human users sign in	OAuth 2.0 / OpenID Connect through an identity provider
Ingest	REST and file upload	Hospitals send event streams	Kafka or Kinesis feeding the same pipeline functions

What makes these changes cheap later: the pipeline is pure functions with no database or web code, the API and worker are stateless, and every component is reached through a small interface. Each change replaces one piece, not the design.

4. Build vs buy

Component	Decision	Reasoning
Mapping, normalization, validation, lineage, matching rules	**Build**	This is the product's core and what makes it trustworthy. It must be fully understood and controlled
Database	**Buy** in production (Amazon RDS)	Backups, failover, patching and encryption are commodity work
Job queue	**Build** on Postgres now, **buy** (SQS) at scale	No extra infrastructure until volume justifies it
HTTPS	**Buy**: Caddy now, a load balancer with ACM certificates later	Certificates and TLS should never be hand-built
Authentication	**Build** API keys now, **buy** an identity provider for human users	Machine-to-machine keys are simple; user login, MFA and SSO are not

Component	Decision	Reasoning
Integration engine (Mirth Connect, Rhapsody)	**Consider buying** when HL7v2 and C-CDA feeds arrive	Connectors for many hospital systems are a large, specialized effort. Mirth Connect became commercial-only with version 4.6 in 2025, with community forks available, so licensing is part of that decision
Patient matching (commercial MPI)	**Consider buying** at scale	Matching errors carry clinical risk, and vendors have years of tuning and reference data
LLM for verification	**Buy** API access under a data-processing agreement, or **self-host** a small model. Never train one	A narrow classification task. Self-hosting keeps patient-adjacent data inside our infrastructure
FHIR server (HAPI FHIR, or a cloud FHIR service)	**Consider** when customers want a standard FHIR API on the output	Faster than building FHIR search and conformance, at the cost of vendor dependence
Error tracking and monitoring	**Buy** (such as Sentry, CloudWatch)	Commodity tooling

5. MVP priorities

Built first: everything that makes the system safe with clinical data. Nothing lost, every change traceable, nothing silently changed, no wrong merges, no cross-org leaks. Those are the properties a healthcare customer would check first, and the hardest to add later.

Item	Status	Why
Raw storage, idempotency, versions, lineage	Built	The foundation of trust and reprocessing
Quarantine and reprocessing	Built	Required, and the core of "never drop data"
Matching with strong-identifier rule and verifier	Built	A wrong merge is the most dangerous failure
Org isolation in the database	Built	A leak between customers ends the business
Tests, including failure and security scenarios	Built	Proof the above actually holds
Console UI and evaluation script	Built	They make the system easy to try and its quality measurable
FHIR-lite instead of full FHIR	Simplified	Full conformance is a large effort that adds little to the core design
Hand-set matching weights	Simplified	Learning weights needs labelled real data
Rule-based verifier; LLM optional	Simplified	Proves the veto design without AI cost or data-handling work
Postgres as the queue; in-memory rate limits	Simplified	No extra infrastructure at this scale
Field-level encryption of identifiers	Left out	Encryption at rest and access control cover the MVP; field-level encryption is the first production addition
Resolution workflow for findings	Left out	Conflicts are recorded and visible; resolving them is a separate product feature

Item	Status	Why
HL7v2 and C-CDA input, blood pressure	Left out	Specialized formats and a two-value observation; CSV and JSON already exercise every normalization rule
Streaming ingest	Left out	Optional bonus; the pipeline functions can be reused by a stream later
Error tracking and alerting	Left out	Commodity tooling to add at deployment

6. Key assumptions

- Sources send structured CSV or JSON, and each can declare its date formats, time zone and codes in a mapping file.
- Each source gives its records a stable id, so corrections can be recognized as new versions.
- Data volume per organization is moderate, up to low millions of records.
- Each organization has someone to review uncertain matches and quarantined records.
- Organizations never share patients with each other through this system; cross-org linking would need consent and a data-sharing agreement.
- All data in this project is synthetic.